Task W10: Trees

Branching Out

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

Overview

This task has seven required problems. Problems 1 through 5 are the same kinds of problems we work through in lecture: counting nodes and leaves, traversing a tree, and tracing insertions and deletions in a binary search tree (BST). Problem 6 asks you to think a step further and rebuild a tree from its traversals; for that one, the explanation is as important as the drawing. Problem 7 introduces storing a binary tree in an array instead of with references.

Use the BST algorithms from our lectures, notes, and book. In particular, do **not** use a balanced-tree algorithm for any problem, and when you delete a node with two children, replace it with its **inorder successor**.

Diagram Submission Instructions

Most problems ask for a diagram rather than typed text. You may choose one of the following options.

- **Hand-drawn on paper or a whiteboard.** Scan or take clear, well-lit photos. Images must be easy to read and clearly labeled with problem numbers. For this task in particular, this is the recommended option: trees are much faster to sketch by hand than to build in software.
- **Hand-drawn using an app** (for example Vittle Pro or Adobe Fresco). Save as an image file (PNG, JPG) or PDF. This is a convenient option but may be harder without a stylus.
- **Created with diagramming software** (for example Draw.io, LibreOffice Draw, or Excalidraw). Save in the tool's native file format; do not export as an image or PDF.
- **Other tools.** If you would like to use a different diagramming application, ask for approval first. Note that I have limited installation access on my office computer.

Include your name and the problem number on every diagram. Answers that are only a list of letters or numbers, or a short written explanation, may be typed.

If you find something that needs to be debugged

If, while working through this assignment, you discover something in the problem statements that needs to be debugged, submit a separate document describing what you found. This is separate from, and in addition to, your answers.

Where to submit

Type your answers in the answer sheet that comes with this task (a Word or OpenDocument file), and save it as .docx or .odt. Put everything for this task (your answer sheet, your diagram file, and any debugging document) in a folder named `taskw10` in your GitHub repository, and commit it.

Conventions

These match the notes. A **level** is counted from the top, with the root at level 1. The **height** of a tree is the number of nodes on the longest path from the root down to a leaf, so a tree with levels 1 through 10 has height 10, a single node has height 1, and an empty tree has height 0.

Problem 1: Nodes, Leaves, and Height

Answer each question about binary trees, and show how you got each answer (a formula with the numbers filled in is enough).

(a) What is the maximum number of nodes in a binary tree of height 7?

(b) What is the maximum number of leaves in a binary tree of height 11?

(c) What is the minimum number of leaves in a binary tree of height 9?

(d) What is the maximum height of a binary tree with 400 nodes?

Problem 2: Traversing a Binary Tree

Figure 1 is a binary tree. Give the order in which each traversal visits the nodes. You only need to list the letters in the correct order.

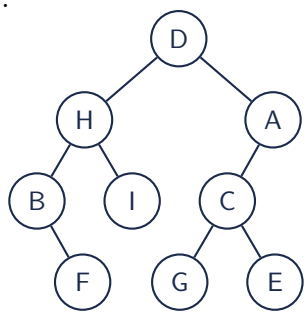


Figure 1: A binary tree with nine nodes, used in Problems 2 and 7. The root is D. D has left child H and right child A. H has left child B and right child I. B has only a right child, F. A has only a left child, C. C has left child G and right child E. F, I, G, and E are leaves.

(a) Preorder

(b) Inorder

(c) Postorder

(d) Breadth-first

Problem 3: Traversing Another Binary Tree

Figure 2 is a binary tree. Give the order in which each traversal visits the nodes. You only need to list the numbers in the correct order.

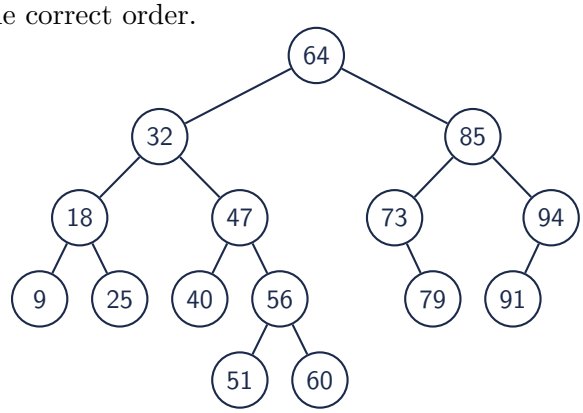


Figure 2: A binary tree with fifteen nodes, used in Problem 3. The root is 64. Its left child is 32, whose children are 18 and 47. 18 has children 9 and 25. 47 has children 40 and 56, and 56 has children 51 and 60. The root's right child is 85, whose children are 73 and 94. 73 has only a right child, 79. 94 has only a left child, 91.

(a) Preorder

(b) Inorder

(c) Postorder

(d) Breadth-first

Problem 4: Inserting into a BST

Starting with an empty BST, insert the following values, reading from left to right. Redraw the tree after each insertion, the way we do in lecture, so your answer is a sequence of nine trees.

50 28 73 41 12 35 89 66 38

When you are done, state the height of the final tree.

Problem 5: Deleting from a BST

Starting with the BST in Figure 3, delete the following values, reading from left to right. Redraw the tree after each deletion. Each deletion starts from the tree left by the one before it; do not start over with the original tree each time. Your first tree will have 14 nodes, and your last tree will have 8 nodes.

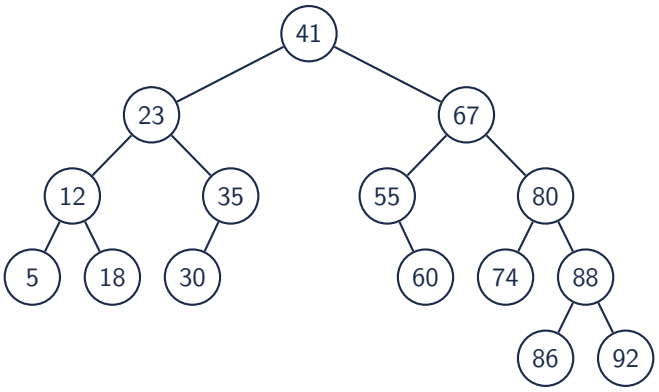


Figure 3: A binary search tree with fifteen nodes, used in Problem 5. The root is 41. Its left child is 23, whose children are 12 and 35. 12 has children 5 and 18. 35 has only a left child, 30. The root's right child is 67, whose children are 55 and 80. 55 has only a right child, 60. 80 has children 74 and 88, and 88 has children 86 and 92.

5 35 41 88 23 55 12

Next to each tree, write which deletion case applied: the node was a leaf, the node had one child, or the node had two children. For every two-child case, also name the inorder successor you used.

Problem 6: Traversal Detective

So far you have gone from a tree to its traversals. This problem goes the other way.

Part A

A binary tree (not a BST) has these two traversals:

Table 1: The two traversals of the tree in Part A.

Traversal	Visit order								
Preorder	J	K	M	N	Q	L	O	P	R
Inorder	M	K	Q	N	J	L	P	O	R

(a) Draw the tree. Only one binary tree has both of these traversals. You only need to draw the finished tree; you do not need a step-by-step trace showing how you built it.

(b) Explain, in a few sentences, how you used the two traversals together. What does the preorder tell you that the inorder does not, and the other way around?

(c) Check your tree by writing its postorder traversal.

Part B

Now suppose you are given a preorder and a postorder instead. A tree has three nodes, X, Y, and Z. Its preorder is X Y Z and its postorder is Z Y X.

(d) Draw two *different* binary trees that both have this preorder and this postorder.

(e) Explain why preorder and postorder together cannot always identify a binary tree, when preorder and inorder can. What kind of node causes the problem?

Part C

This time the tree is a binary search tree, and you are given only its preorder:

45 20 10 30 25 70 60 65 80

(f) Draw the BST. As in (a), you only need the finished tree, not a step-by-step trace of inserting each value.

(g) In Part A you needed two traversals. Explain why one traversal is enough here.

Problem 7: A Binary Tree in an Array

Section 4 of the notes describes two ways to store a binary tree in an array instead of with references. Use the tree in Figure 1 for both parts.

(a) **Array of nodes with child indices.** Store the nodes in breadth-first order, starting with the root at index 0. Make a table with four columns: index, item, left child index, and right child index. Use -1 when there is no child.

(b) **Position-based array.** Store the same tree with the root at index 0, the left child of index i at $2i + 1$, and the right child at $2i + 2$. Show the array with the index of every slot, and mark each empty slot clearly. How many slots does the array need, and how many of them are empty?

What to Submit

Typed answers go in the answer sheet, and drawings go in your diagram file.

- Problem 1: four answers, each with its work.
- Problems 2 and 3: four traversal orders each.
- Problem 4: nine trees, one after each insertion, and the final height.
- Problem 5: seven trees, one after each deletion, each labeled with its case (and the inorder successor for two-child cases).
- Problem 6: two drawn trees for Part A and Part C, two drawn trees for Part B, a postorder traversal, and three short explanations.
- Problem 7: one table and one array, with the slot counts.
- Any debugging document, if you found something that needs to be fixed.

Everything above goes in the `taskw10` folder in your GitHub repository.